



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

The Payment Pipeline

The senior engineer answer sheet

One payment. Five deliberate failures. One effect, or the design is wrong.

No single feature makes a pipeline reliable. Kafka did not solve the client retry. Idempotency did not solve partition ownership. Offsets did not solve the dual write.

Reliability came from the OVERLAP, which is why this series had eight episodes before this one and not one.

This sheet is the design, then the same design broken five ways on purpose. Sources are named beside each claim, and most of them point back at an earlier episode sheet where the claim was established.

In scope

- Starting from invariants rather than from Kafka
- The write path, in one transaction
- Five failures, and what survives each
- Where ordering actually comes from
- The failure path as architecture, not as a catch block
- What is invisible while everything is green

Out of scope

- Cluster sizing, partition count planning and capacity
- Provider specific payment APIs
- Reconciliation and settlement, which are their own subject
- Multi region and disaster recovery

A design that only describes the success path is a description of a demo.

01 Worked assumptions

Payments in the walkthrough	1
Deliberate failures applied to it	5
Effects that must result	exactly 1
Partitions	6
Partition key	account id
Consumer instances	3, so 2 partitions each

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 Start with what must never be false

Not with Kafka. With the invariants: money is never created or destroyed, one logical payment moves value once, every movement is auditable, and two payments for the same account do not interleave incorrectly.

Every component below exists to protect one of those. Anything that protects none of them is decoration.

Candidates who open with the technology have skipped the part being assessed. The technology is an implementation of the invariants, and it should be introduced as one.

Stating the invariants before the components is what allows each component to be justified or removed, rather than accumulated.

First principles, not vendor documentation

A double entry ledger enforces conservation of value structurally, by requiring debits and credits to balance, rather than by convention.

First principles, not vendor documentation

04 The write path, in one transaction

Claim the idempotency key. Record the ledger movement. Enqueue the outbox row. One transaction, or none of it happened.

The claim is an INSERT against a unique constraint, so it wins or loses atomically and never reads first.

The outbox row is in the same transaction as the movement, which is what removes the dual write. Nothing is published from inside the request.

A unique constraint ensures the values in a group of columns are unique among all rows, which is what makes the claim atomic without a prior read.

PostgreSQL 17, Constraints

Writing the business row and the event row in one database transaction removes the dual write, because only one transactional resource is involved.

PostgreSQL 17, Transactions

05 Failures one and two: the client retries, then the consumer crashes

THE CLIENT RETRIES. The first request claims the key and wins, and moves the money. The retry claims the same key and loses, and returns the stored result. Survived by idempotency alone. Kafka was not involved at all.

THE CONSUMER CRASHES AFTER THE EFFECT. The work succeeded and the commit never landed. After restart the consumer resumes from the last committed offset and sees the record again. The claim already exists, so there is no second effect.

That second one is survived by offsets AND idempotency together. Neither alone would have done it, and saying so is the answer.

Kafka guarantees at-least-once delivery by default, so a consumer that processes a record and fails before committing will see that record again.

Kafka 4.3, Design: Message Delivery Semantics

Committed offsets determine what replays; an idempotency claim determines whether the replay has an effect. The two protect different halves.

First principles, not vendor documentation

06 Failure three: the group rebalances mid batch

A partition is revoked in the middle of a batch and handed to another consumer, which replays from the last commit.

The claim is already there. The replay produces no second effect.

Note what was NOT done: the rebalance was never prevented. It was made HARMLESS, which is a different goal and an achievable one. Preventing rebalances is not available; surviving them is.

Partitions are divided among the members of a consumer group so each partition is consumed by exactly one member, and a membership change redistributes that ownership.

Kafka 4.3, Design: Message Delivery Semantics

Designing for a rebalance to be harmless is achievable, whereas designing for rebalances not to occur is not.

First principles, not vendor documentation



07 Failure four: the database is unavailable

Three questions, in order. Can waiting change this? Yes, it is transient, so retry with backoff. Is the work safe to repeat? Yes, the claim holds. How late may it be? That is a business question and it sets the exit condition.

Backoff without idempotency would have multiplied the damage rather than absorbing it.

This is the clearest example in the whole series of two episodes only working together.

A retry is useful only where a relevant condition can differ on the next attempt, which is what makes a transient dependency failure retryable and a malformed payload not.

First principles, not vendor documentation

Retry with backoff is safe only where the operation is idempotent; otherwise each additional attempt is an additional effect.

First principles, not vendor documentation

08 Failure five: the outbox publisher dies

Everything is healthy. Nothing is lost. Everything is late.

The database is fine because it is committing. Kafka is fine because it is up. The consumers are fine because they have nothing to consume.

Only the age of the oldest unpublished row can see this, and it is the one panel most teams do not have.

A stopped relay between a database and Kafka produces no error in either system, because neither is failing.

First principles, not vendor documentation

The age of the oldest unpublished outbox row is the only local signal that detects a stopped relay.

First principles, not vendor documentation

09 Ordering holds exactly where the key put it

Same key, same partition, ordered. Different keys, possibly different partitions, and no ordering promise between them at all.

The design chose the key. The key chose the ordering. Nothing else did, and no amount of configuration adds ordering the key did not buy.

So the interview question behind every ordering question is: what is your key, and why that one?

Kafka provides a total order over records within a partition, and no ordering guarantee between records in different partitions.

Kafka 4.3, Topics and Logs

Records with the same key are assigned to the same partition under the default partitioner, which is what makes per-key ordering a design choice rather than a configuration option.

Kafka 4.3, Producer Configs

10 The failure path is architecture

It has its own topics, its own traffic and its own signals: retry topics with delays, a dead letter topic carrying enough context to replay, a path back into the main flow, and separate monitoring for all of it.

A design reviewed only on its happy path has not been reviewed.

And the replay path carries the subtlest condition in the whole series: a replay is safe only if the idempotency identity outlived the record.

A dead letter topic is an ordinary Kafka topic; nothing in the broker treats it specially and the meaning is a convention of the application.

Kafka 4.3, Topics and Logs

A replay is safe only where the idempotency identity for that operation still exists; if claims have been cleaned up, the replay is a new first attempt.

First principles, not vendor documentation

11 All five, scored

CLIENT RETRY: a stable identity prevented a second payment. Idempotency, on its own.

CONSUMER CRASH: committed progress replayed and the claim absorbed it. Offsets plus idempotency.

REBALANCE: ownership changed and the claim absorbed it again. Group semantics plus idempotency.

DATABASE DOWN: classified transient, retried with backoff, safe because of the claim. Retry policy plus idempotency.

PUBLISHER DIED: nothing lost, everything late, and only row age saw it. Outbox plus observability.

One payment. Five failures. One effect. And idempotency appears in four of the five rows, which is the finding rather than the summary.

Every failure above is survived by a combination rather than by a single mechanism, which is the argument against treating any one of them as the reliability feature.

First principles, not vendor documentation

Questions to ask before you answer

1. What must never be false in this system?
2. What identifies one logical operation, and who generates it?
3. What is inside the one transaction, and what is outside it?
4. What is the partition key, and what ordering does it buy?
5. Which failures retry, and which never do?
6. What happens to in flight work when a partition is revoked?
7. What is invisible while every component reports healthy?
8. What would you deliberately NOT build for this workload?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



X

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.